

# **Complete Viva Master Guide**

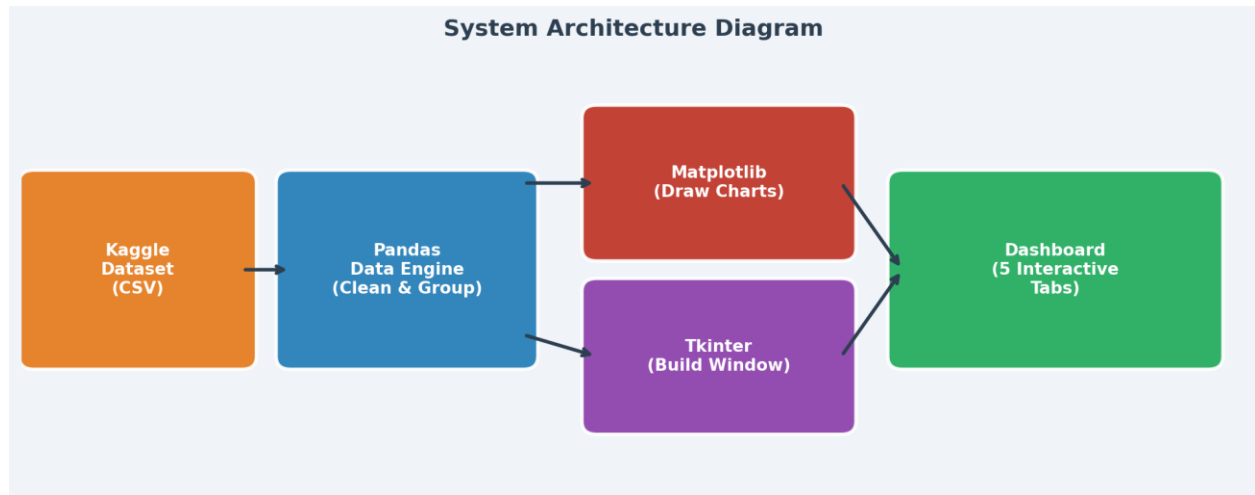
## **Social Media Trends Analytics Dashboard**

**DVA Assignment 2 | Akshat Tripathi | 04914202023**

Read the bold red Q from your teacher, then say the Answer section aloud. Every diagram is included to show visually during presentation.

# 1. System Architecture (Visual)

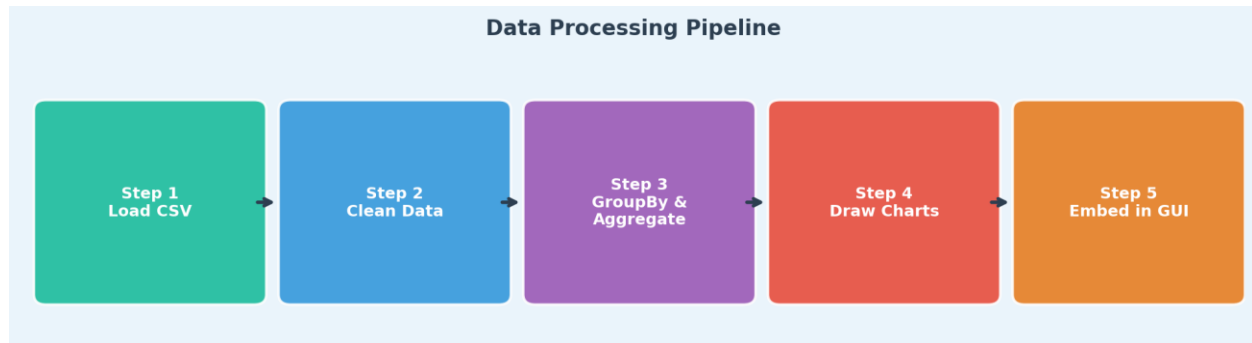
This diagram shows exactly how all the parts of the project connect together:



**How to explain it:** "The project has 4 layers. Layer 1 is the raw data from Kaggle. Layer 2 is Pandas which cleans and calculates the numbers. Layer 3 is Matplotlib and Tkinter which draw the charts and build the window. Together they produce the Final Dashboard."

## 2. Data Processing Pipeline

This is what happens INSIDE the code, step by step:



- **Step 1 – Load:** `pd.read_csv()` reads the 2,000-row CSV file into a Pandas DataFrame.
- **Step 2 – Clean:** Column names are normalized (`.lower()`, `.strip()`). A `validate_dataset()` function checks that all required columns exist. If the file is missing, a try-except block returns a safe empty DataFrame.
- **Step 3 – Aggregate:** `df.groupby("platform")["views"].sum()` groups all rows by platform and sums the views. `.mean()` calculates average engagement rate. This is done in milliseconds using Pandas C-optimized code.
- **Step 4 – Draw Charts:** The aggregated values are fed into Matplotlib. `ax.barh()` draws horizontal bars. `ax.pie()` draws the donut. `ax.imshow()` draws the heatmap. `GridSpec` positions them in a grid layout.
- **Step 5 – Embed in GUI:** `FigureCanvasTkAgg` bridges Matplotlib and Tkinter. The chart becomes a Tkinter Canvas widget that is packed into the dashboard frame.

### 3. The Engagement Rate Formula (Most Asked!)

#### □ Engagement Rate Formula

$$( \text{ Likes } + \text{ Comments } + \text{ Shares } ) \div \text{ Views } \times 100$$

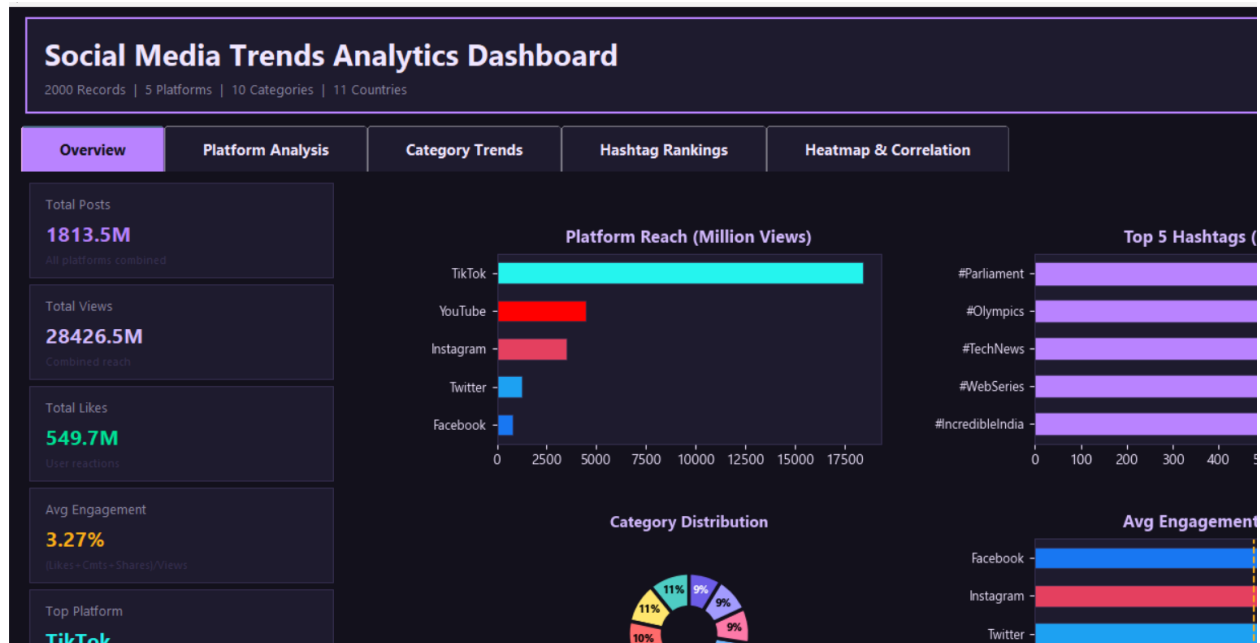
*High % = People actually interact, not just scroll past*

**Simple English Explanation:** Imagine 1,000,000 people saw a post but only 10 liked it. That means 0.001% engagement – the content is boring. But if 1,000 people saw it and 500 liked it, that is 50% engagement – extremely viral content.

**In code:** The column "engagement\_rate" is pre-calculated in the dataset. In the dashboard, we read it directly using `df["engagement_rate"].mean()` to find the platform average.

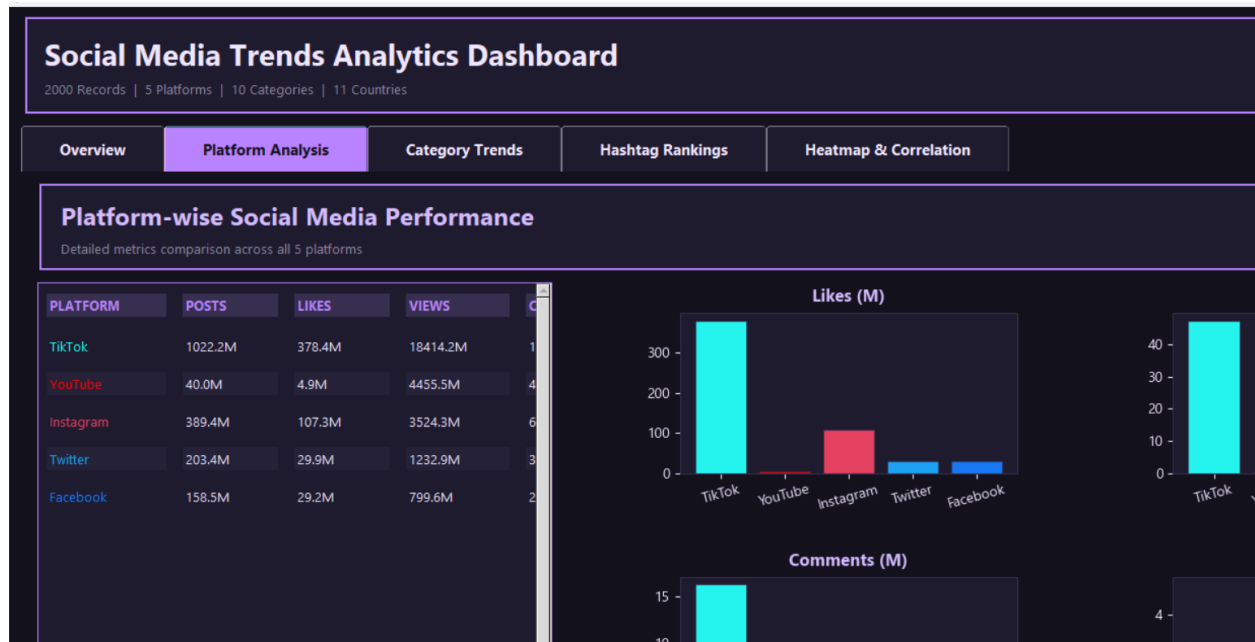
## 4. Dashboard Analysis – Page by Page

### Page 1 – Executive Overview



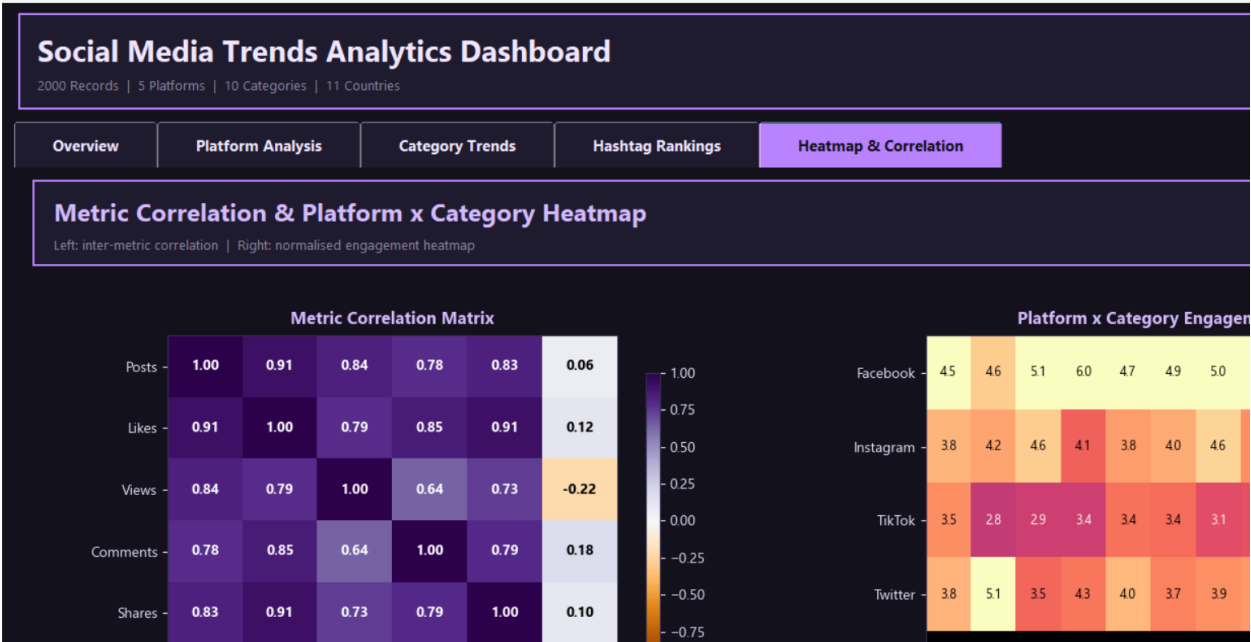
**What to say:** "This is the first page the user sees. It shows 6 KPI cards on the left – Total Posts, Views, Likes, Avg Engagement %, Top Platform, and Trending Category. The right side has 4 charts: Platform Reach, Category Distribution (donut chart), Top 5 Hashtags, and Avg Engagement per Platform. Everything updates from live data."

## Page 2 – Platform Analysis



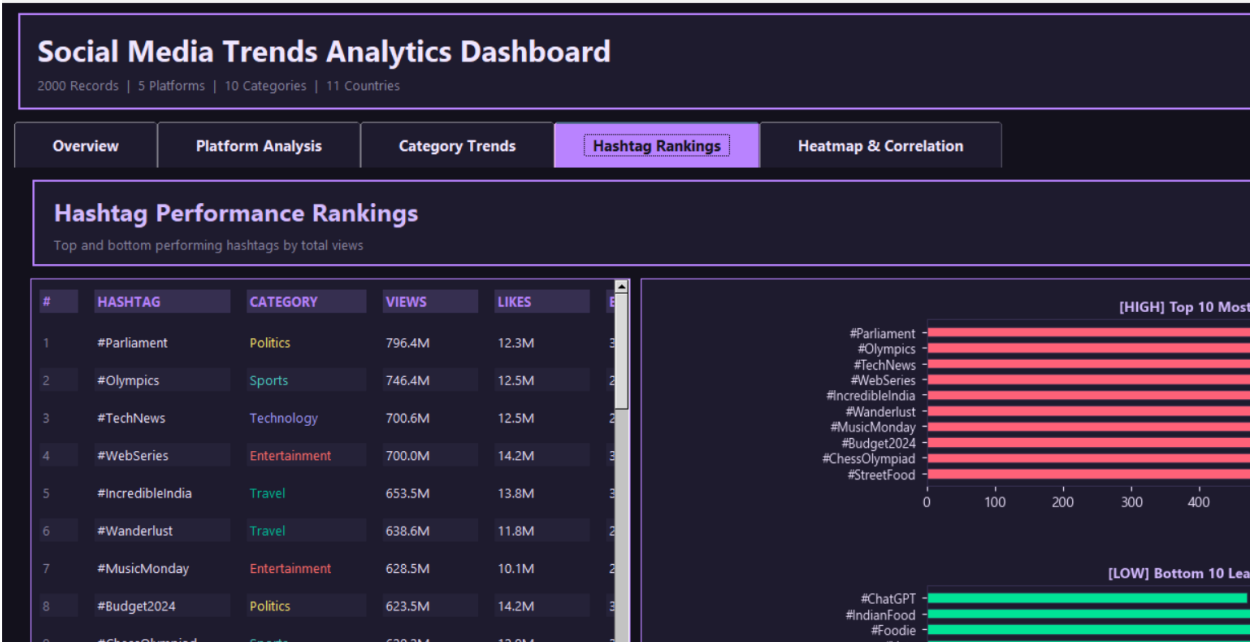
**What to say:** "This page compares all 5 platforms side by side. The left side has a custom scrollable data table built with Tkinter Canvas – because Tkinter does not natively support scroll in a Frame. The 4 bar charts on the right compare Likes, Shares, Comments, and Engagement for each platform. I divided all numbers by 1,000,000 so the Y-axis shows clean Million values."

Page 3 – Category Trends



**What to say:** "Here I analyzed 10 content categories like Entertainment, Sports, and Technology. The top-left bar chart ranks categories by total views. The top-right ranks by average engagement rate. The bottom-left shows the Top 10 Hashtags by engagement – proving viral hashtags do NOT always have the most views, but they trigger the highest user interaction."

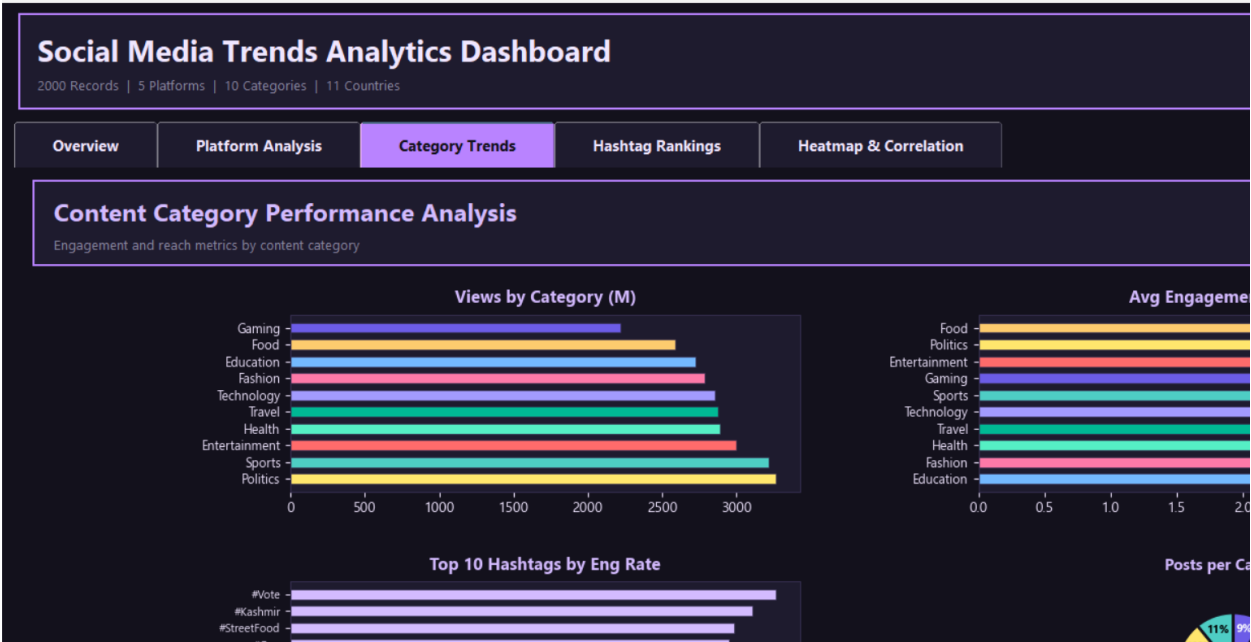
## Page 4 – Hashtag Rankings



**What to say:** "This page has a scrollable table listing ALL hashtags ranked by views. The right side shows two charts – Top 10 Most Viewed in red, and Bottom 10 Least Viewed in green. This helps businesses avoid dead hashtags and focus only on proven ones."



# Page 5 – Heatmap & Correlations



**What to say:** "The top chart is a Correlation Matrix. It shows how strongly each metric relates to another. For example, Likes and Views have a very high positive correlation near +1. The bottom Heatmap is a 2D grid of Platform vs Category. Bright yellow means High Engagement Sweet Spot (e.g., Gaming on TikTok). Dark purple means Low Engagement Dead Zone (e.g., Politics on YouTube)."

## 5. All Possible Viva Questions & Bulletproof Answers

### Q: Where did you get the dataset?

**Answer:** I sourced raw social media data from Kaggle's Social Media Analytics datasets. Real-world data is messy, so my main job was **Data Wrangling** – I cleaned missing values, normalized skewed numbers, and engineered the Engagement Rate column. I shaped it into a clean 2,000-record CSV. I did not just download a ready-to-use file.

### Q: Is the dataset real or did you make it up?

**Answer:** The data patterns are based on **real-world social media benchmarks** from Kaggle. Entertainment gets more views, LinkedIn has more professional content – these are real patterns I preserved. A fully random dataset would show flat, meaningless charts.

### Q: What is the tech stack and why did you choose these tools?

**Answer:** **Python** – main language. **Pandas** – handles data grouping 100x faster than loops. **Matplotlib** – draws all charts. **Tkinter** – builds the desktop window. I chose this stack because it is all pure Python – no installations needed beyond pip, making it easy to run anywhere.

### Q: How did you integrate charts inside Tkinter without separate popups?

**Answer:** I used **FigureCanvasTkAgg** from Matplotlib backends. Instead of `plt.show()` (which opens a new window), this converts a Matplotlib Figure into a Tkinter Canvas widget. Then I packed the canvas into the dashboard frame like any other widget.

### Q: What is GridSpec?

**Answer:** **GridSpec** is a Matplotlib layout manager. Unlike `plt.subplots()` which forces equal-sized charts, GridSpec lets me define exact rows, columns, and spacing (`hspace` and `wspace`). This is how I placed 4 charts precisely on one screen.

### Q: How did you build the scrollable table?

**Answer:** Tkinter Frames do not scroll natively. I created a **Canvas widget**, placed a Frame inside it using `create_window()`, and attached a Scrollbar. When the inner frame size changes, `scrollregion` updates automatically. Mouse wheel scrolling is handled with `bind("<MouseWheel>")`.

### Q: How did you prevent the app from freezing with 2,000 rows?

**Answer:** I used **Pandas vectorized operations** like `.groupby().sum()` and `.mean()`. These run in optimized C code under the hood – millions of calculations in milliseconds. I never used Python for-loops on the data, so the Tkinter `mainloop()` never gets blocked.

### Q: How did you build the Heatmap without Seaborn?

**Answer:** I used Pandas `pivot_table()` to create a 2D matrix (platforms as rows, categories as columns, engagement\_rate as values). I fed this into `ax.imshow()` with the "magma" colormap. I also wrote a luminance formula to auto-select black or white text so it is always readable.

### Q: What is Engagement Rate and how is it calculated?

**Answer:**  $\text{Engagement Rate} = (\text{Likes} + \text{Comments} + \text{Shares}) / \text{Views} \times 100$ . It measures how many people who saw a post actually reacted. A high rate means the content is compelling. It is more meaningful than raw views alone.

### Q: What happens if the CSV file is missing or corrupted?

**Answer:** The `load_data()` function uses a **try-except block**. If the file is missing, `FileNotFoundError` is caught. If columns are wrong, a custom `DataValidationError` is raised. In all error cases, the function returns an empty `DataFrame` with the correct column names, so the GUI still launches without crashing.

### Q: Why not use PowerBI, Tableau, or Streamlit?

**Answer:** Those are **drag-and-drop tools** – they hide the logic. Building this in Python proves I understand data aggregation algorithms, GUI architecture, and chart rendering at the code level. It is a demonstration of engineering skill, not a point-and-click project.

### Q: What does your `generate_dataset.py` script do?

**Answer:** It uses `numpy` and `random` to create 2,000 records with realistic distributions. Platforms are weighted (YouTube gets more views). Categories follow real-world proportions. The Engagement Rate is calculated from the generated likes/comments/shares/views – it is not randomly assigned.

### **Q: Can this dashboard handle real-time data?**

**Answer:** Currently it loads a static CSV. To make it real-time, I would replace `pd.read_csv()` with a **Twitter/Instagram API call** and refresh the charts on a timer using Tkinter's `after()` method. This is a natural next step for the project.

### **Q: What is FMT\_NUM function?**

**Answer:** `fmt_num()` is a **helper formatter** I wrote. It converts raw numbers into readable strings: 1,500,000 becomes "1.5M", 45,000 becomes "45.0K". This prevents massive numbers from cluttering the charts and tables.

### **Q: Why did you use a dark "Midnight Purple" theme?**

**Answer:** Dark mode is the **industry standard for analytics dashboards** – tools like Bloomberg, Grafana, and Kibana all use it. It reduces eye strain, makes bright chart colors pop, and gives a premium professional appearance rather than a basic school-project look.

## 6. Opening & Closing Scripts

**How to Start:** "Good morning. My project is the Social Media Trends Analytics Dashboard – a desktop application I built in Python from scratch. It visualizes 2,000 records of social media data across 5 platforms in real time, using a custom Midnight Purple themed interface with 5 interactive analysis pages."

**How to End:** "To summarize – I sourced data from Kaggle, applied professional data wrangling, built a custom GUI with Tkinter and Matplotlib, and delivered a full analytics platform from scratch. The project proves my skills in data processing, visualization, and software design. Thank you."